

LAB 4

MỤC TIÊU

Kết thúc bài thực hành sinh viên có khả năng:

- ✓ Hiểu cách sử dụng Networked Properties để đồng bộ hóa dữ liệu trong môi trường mạng.
- ✓ Đồng bộ trạng thái của Player (HP, MP, ...).
- ✓ Quản lý Animation của Player để hiển thị nhất quán trên các client.
- ✓ Cập nhật trạng thái Animation bằng Network Properties

NỘI DUNG

BÀI 1:

Sinh viên cần thực hiện:

- Sử dụng Networked Properties để lưu trữ và đồng bộ HP và MP giữa các client.
- Cập nhật giao diện hiển thị HP và MP khi chúng thay đổi.

BÀI 2:

Sinh viên cần thực hiện:

- Thêm Animator vào nhân vật.
- Sử dụng Networked Properties để đồng bộ trạng thái animation.

BÀI 3:

Sinh viên cần thực hiện:

- Thêm các trạng thái animation nâng cao (Attack, Hurt, Jump).
- Sử dụng Networked Properties để đồng bộ trạng thái nâng cao.

BÀI 4: GV CHO THÊM

***** YÊU CẦU NỘP BÀI:**

Sv nén file bao gồm các yêu cầu đã thực hiện trên, nộp lms đúng thời gian quy định của giảng viên. Không nộp bài coi như không có điểm.

--- Hết ---

HƯỚNG DẪN

BÀI 1:

Bước 1: Cập nhật Prefab Player

- ✓ Thêm script quản lý dữ liệu HP, MP
- ✓ Tạo script PlayerStats và gắn vào Prefab Player

```
public class PlayerStats : NetworkBehaviour
{
    [Networked] public int HP { get; set; }
    [Networked] public int MP { get; set; }
    private void Start()
    {
        if (Object.HasInputAuthority)
        {
            HP = 100; // Mặc định HP đầy đủ
            MP = 50;  // Mặc định MP đầy đủ
        }
    }
    private void Update()
    {
        if (Object.HasInputAuthority)
        {
            if (Input.GetKeyDown(KeyCode.H)) HP -= 10; // Giảm HP khi nhấn H
            if (Input.GetKeyDown(KeyCode.M)) MP -= 5;  // Giảm MP khi nhấn M
        }
    }
}
```

Bước 2: Hiển thị HP, MP trên giao diện người dùng

- ✓ Tạo UI: Tạo một Canvas, thêm Text UI để hiển thị HP và MP.
- ✓ Cập nhật UI khi dữ liệu thay đổi: Tạo script PlayerUI và gắn vào Canvas

```
public class PlayerUI : MonoBehaviour
{
    public TextMeshProUGUI hpText;
    public TextMeshProUGUI mpText;
    private PlayerStats playerStats;

    private void Start()
    {
        playerStats = FindObjectOfType<PlayerStats>();
    }

    private void Update()
    {
        if (playerStats != null)
        {
            hpText.text = $"HP: {playerStats.HP}";
            mpText.text = $"MP: {playerStats.MP}";
        }
    }
}
```

BÀI 2:

Bước 1: Cập nhật Prefab Player

- ✓ Thêm Animator: Tạo Animator Controller cho nhân vật với các trạng thái:
 - Idle
 - Run
- ✓ Thêm điều kiện chuyển trạng thái dựa trên IsRunning.
- ✓ Cập nhật script PlayerStats: Thêm trạng thái chạy vào nhân vật

```
[Networked] public bool IsRunning { get; set; }
private void Update()
{
    if (Object.HasInputAuthority)
    {
        IsRunning = Input.GetAxis("Horizontal") != 0
            || Input.GetAxis("Vertical") != 0;
    }
}
```

Bước 2: Đồng bộ trạng thái với Animator

- ✓ Tạo script PlayerAnimator, gắn vào Prefab Player.

```
public class PlayerAnimator : NetworkBehaviour
{
    private Animator animator;
    private PlayerStats stats;

    private void Start()
    {
        animator = GetComponent<Animator>();
        stats = GetComponent<PlayerStats>();
    }

    private void Update()
    {
        animator.SetBool("IsRunning", stats.IsRunning);
    }
}
```

BÀI 3:

Gợi ý

Cập nhật Animator Controller:

- ✓ Thêm các trạng thái: Attack, Hurt, Jump.
- ✓ Tạo các trigger tương ứng trong Animator.
- ✓ Cập nhật script PlayerStats: Thêm trạng thái IsAttacking, IsHurt, và IsJumping:

Đồng bộ trạng thái với Animator:

- ✓ Cập nhật script PlayerAnimator
- ✓ Cập nhật các trạng thái của các biến quản lý animator